Name: Shryitha Suddala

Hall Ticket:2303A51637

Batch:29

Assessment:10.2

Task 1: Error Detection and Correction

Prompt:

Correct the below code with breif explanation of the identifed syntax error and logical error

Code:

```
#Correct the below code  with breif explanation of the identifed syntax error and logical error
"""def calculate_total(nums)
sum = 0
for n in nums
sum += n
return total"""
#Syntax Error: The code does not have any syntax errors.
#Logical Error: The code correctly calculates the total sum of the numbers in the list. However, it does not handle the case where the input list is empty.
# In such a case, it will return 0, which may be acceptable, but it could be more informative to return a message indicating that the list is empty.
#  Additionally, using 'sum' as a variable name is not recommended as it shadows the built-in 'sum' function in Python.
# It would be better to use a different variable name, such as 'total'.
#Corrected Code:
def calculate_total(nums):
    if not nums:
        return "The list is empty."
    total = 0
    for n in nums:
        total += n
    return total
#Example usage:
numbers = [1, 2, 3, 4, 5]
print(calculate_total(numbers))  # Output: 15
print(calculate_total([]))       # Output: The list is empty.
```

Output:

```
PS C:\Users\shryi\Downloads\AI> & C:/Users/shryi/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/shryi/Downloads/AI/task 10.2.py"
15
The list is empty.
PS C:\Users\shryi\Downloads\AI>
```

Significance:

This task helps students learn how to identify and fix **syntax errors** (missing colons, indentation, etc.) and **logical errors** (wrong variables, incorrect output). It improves debugging skills and teaches how AI can assist in making code executable and correct. It also builds confidence in analyzing faulty programs.

Task 2: Code Style Standardization

Prompt:

write a well structured ,consistently formatted Python code with standard style conversion

Code:

```
#def findSum(a,b):return a+b print(findSum(5,10)),write a well structured ,consistently formatted Python code with standard style conversion
def find_sum(a, b):
    """Returns the sum of two numbers."""
    return a + b
# Example usage:
result = find_sum(5, 10)
print(result)  # Output: 15
```

Output:

```
PS C:\Users\shryi\Downloads\AI> & C:/Users/shryi/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/shryi/Downloads/AI/task 10.2.py"
15
PS C:\Users\shryi\Downloads\AI>
```

Significance:

This task focuses on writing **clean, readable, and professional Python code** following standard conventions (PEP-8). It helps students understand the importance of proper formatting, spacing, naming, and structure. Using AI here demonstrates how automated tools can improve code quality and maintainability.

Task 3: Code Clarity Improvement

Prompt:

Rewrite the code with proper function,variable names,proper indentation and improved clarity

Code:

```
#def f(x,y):return x-y*2 print(f(10,3)),rewrite the code with proper function,variable names,proper indentation and improved clarity
def calculate_difference(x, y):
    """Returns the difference of x and twice y."""
    return x - (y * 2)
# Example usage:
result = calculate_difference(10, 3)
print(result)  # Output: 4
```

Output:

```
4
PS C:\Users\shryi\Downloads\AI> & C:/Users/shryi/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/shryi/Downloads/AI/task 10.2.
4
PS C:\Users\shryi\Downloads\AI> =
```

Significance:

This task improves program readability by replacing unclear function and variable names with meaningful ones. It teaches that code should be **easy to understand**, not just correct. Clear code helps future developers (and yourself) quickly understand logic, reducing errors during maintenance.

Task 4: Structural Refactoring

Prompt:

Refactor the following Python code by removing repetition and converting it into reusable functions. Follow modular programming principles and keep the code clean and readable.

Code:

```python
#Refactor the following Python code by removing repetition and converting it into reusable functions. Follow modular programming principles and keep t
"""Sample Input Code:
print("Hello Ram")
print("Hello Sita")
print("Hello Ravi")"""
#Refactored Code:
def greet(name):
    print(f"Hello {name}")
# Example usage:
greet("Ram")
greet("Sita")
greet("Ravi")
```

Output:

```
PS C:\Users\shryi\Downloads\AI> & C:/Users/shryi/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/shryi/Downloads/AI/task 10.2.py"
Hello Ram
Hello Sita
Hello Ravi
PS C:\Users\shryi\Downloads\AI>
```

Significance:

This task introduces **modular programming** by removing repetition and creating reusable functions. It shows how AI can help refactor duplicated code into cleaner structures, making programs more scalable and organized. It also promotes the DRY principle (Don't Repeat Yourself).

Task 5: Efficiency Enhancement

Prompt:

Optimized the below python code

Code:

```
#Optimized the below python code
"""numbers = [ ]
for i in range(1, 500000):
numbers.append(i * i)
print(len(numbers))"""
#Optimized Code:
numbers = [i * i for i in range(1, 500000)]
print(len(numbers))
```

Output:

```
PS C:\Users\shryi\Downloads\AI> & C:/Users/shryi/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/shryi/Downloads/AI/task 10.2.py"
499999
PS C:\Users\shryi\Downloads\AI>
```

Significance:

This task teaches performance optimization by converting slow loops into faster approaches like list comprehensions. It highlights how AI can improve execution speed and memory usage. Students learn to write **efficient Python programs**, which is important for real-world applications with large data.